

# CS6811 – Project Work

## Federated Learning for Code Generation with Hallucination-Aware Fine Tuning

### Team Members:

Abhinavh P (2022503059)

Prabhakara Arjun R (2022503003)

Buvanes Srivardan K (2022503037)

### MENTOR

Dr. B. Thanasekhar,  
Professor,  
Computer Technology,  
MIT

# INTRODUCTION

- Large Language Models (LLMs) have remarkable capabilities in code generation ,enabling automated solutions to a wide range of software engineering tasks.
- However LLMs frequently suffer from hallucinations in code generation, producing syntactically valid but semantically incorrect programs, non-existent APIs, or logically flawed implementations.
- Existing approaches to mitigating code hallucinations primarily rely on prompt engineering, repeated sampling, or full model fine-tuning.
- Full fine-tuning is computationally expensive and impractical in low-resource or decentralized settings, and repeated sampling does not address the underlying causes and mitigation of hallucination.
- In real-world development environments, code generation systems are often deployed across multiple decentralized clients where data cannot be centrally aggregated due to privacy, ownership.
- To address these limitations we propose a hallucination-aware code generation framework that combines program analysis, targeted automatic program repair, and federated parameter-efficient fine-tuning using LoRA.

# PROBLEM STATEMENT

Code generation using large language models (LLMs) is currently hindered by hallucinations. Existing mitigation strategies typically rely on supervised fine-tuning or reinforcement learning from human feedback (RLHF), where models are trained to align with human preferences. While parameter-efficient fine-tuning methods such as LoRA reduce computational overhead compared to full fine-tuning, both approaches primarily optimize model behavior but without explicit verification of program correctness. As a result, these methods may improve correctness but often fail to systematically detect and correct deeper structural, semantic, and execution-level errors in generated code.

Furthermore, in real world scenarios, high-quality code and execution feedback are distributed across multiple organizations making centralized data collection infeasible due to privacy concerns. Without a secure and decentralized learning mechanism, models are unable to effectively learn from diverse hallucination patterns observed across clients. This creates a critical gap for a unified framework that integrates explicit hallucination detection and automated program verification with privacy-preserving, parameter-efficient adaptation to improve the quality of code generation.

# OBJECTIVES

- To. Develop a federated learning framework for LLM-based code generation using LoRA, enabling decentralized training across clients while preserving privacy and reducing computational overhead.
- Design a hallucination-aware pipeline that detects and classifies code errors using static analysis (AST, LIB/API) and dynamic execution, forming a structured fault taxonomy.
- Propose Fed-Error Average, an error-aware aggregation strategy that weights client updates based on hallucination error distributions to handle heterogeneous, non-IID data.
- Evaluate the framework on HumanEval, MBPP, and DS-1000 using Pass@1 and CodeBLEU metrics to validate improvements in functional correctness and hallucination reduction.

# ARCHITECTURE DIAGRAM

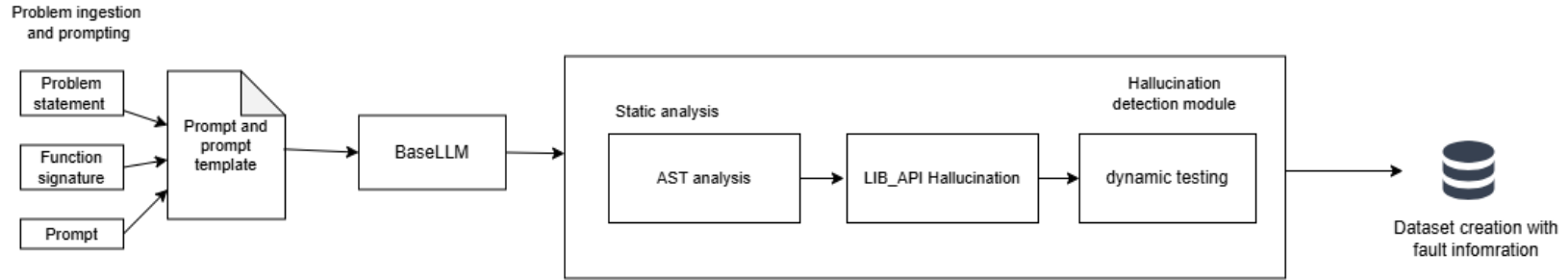


Fig1. Client side hallucination detection and mitigation framework

# ARCHITECTURE DIAGRAM

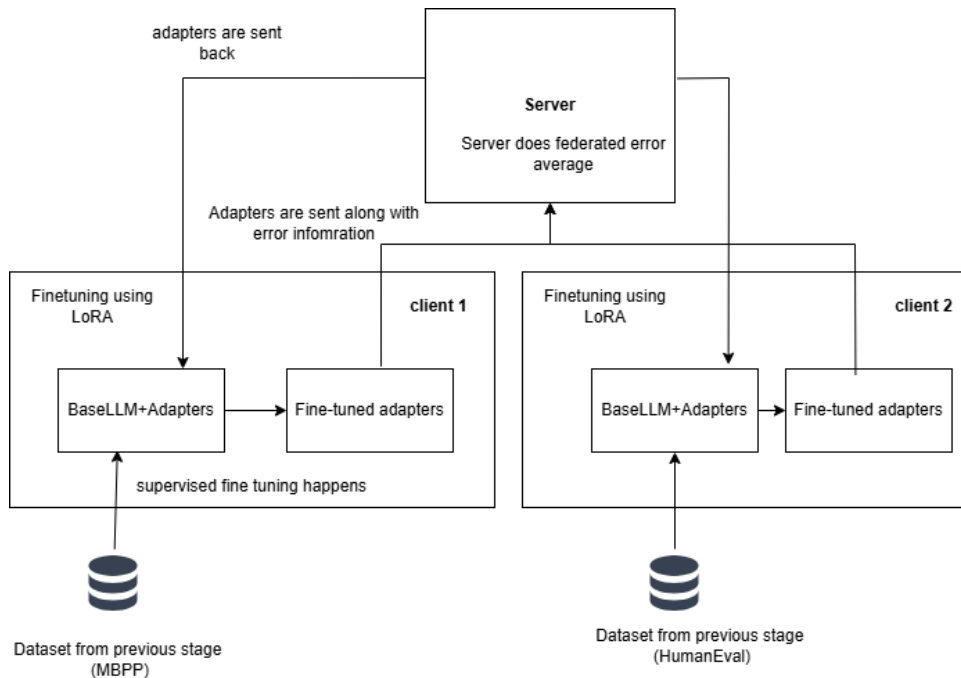


Fig 2. Federated learning + LoRA for fine-tuning

# METHODS/MODULES USED

## 1.Problem ingestion and code generation

**Input:** Benchmark code-generation datasets.

**Output:**Initial LLM-generated code solutions (potentially containing hallucinations).

### Methods:

- **Dataset Loading:**Benchmark code-generation datasets including MBPP, HumanEval, and DS-1000 are loaded using the Hugging Face library.
- **Client-wise Dataset Allocation:**Each federated client is assigned a distinct dataset or dataset subset to simulate heterogeneous behaviour.
- **Problem Specification Construction:**For each task, structured inputs consisting of the problem statement, function signature, constraints is constructed.
- **Single-Shot Prompting:**Code generation is performed using a single-shot prompting strategy, where the model receives the task specification without iterative refinement.
- **Baseline Inference:**The frozen Qwen2.5-Coder-1.5B pretrained model is used to generate initial code outputs, which may contain hallucinations and serve as baseline for downstream analysis.

## 2.Hallucination detection

**Input:** LLM-generated code obtained from the code generation module.

**Output:**Detected hallucination types along with structured diagnostic information.

### **Methods:**

- **AST Construction and Analysis:** Abstract Syntax Tree (AST) is constructed and analyzed to detect syntax errors, undefined variables, and invalid language constructs.
- **Library analysis:** We use ast to check if each methods,functions align with the library in static setup.
- **Dynamic Analysis:** The test cases are executed on the code to detect runtime errors and semantic hallucinations.
- **Hallucination Labeling:** Detected errors are categorized into different hallucination types, and the resulting information is recorded for dataset construction during fine-tuning.



### 3.Dataset creation

**Input:** Buggy code, detected hallucination type, fault information.

**Output:** Final dataset for fine-tuning using LoRA at each client.

**Methods:**

- **Dataset Construction:** Aggregates task-level records containing dataset name, task ID, prompt, hallucination status, AST/dynamic/library analysis, generated code, canonical solution, error sources, types, and line information.
- **APR Limitation Handling:** Although an APR module was explored for fixing hallucinated code using fault signals, it was not used due to unreliable patch generation from the Qwen coder model.
- **Target Selection Strategy:** If code passes hallucination checks then use generated code as target; otherwise use canonical benchmark solution as target to ensure correctness.
- **Supervised Pair Formation:** Prompts and corresponding correct solutions form input–output pairs for LoRA fine-tuning, ensuring consistent and accurate supervision. Evaluation Objective: Focus is on hallucination reduction in a federated setting, with correctness validated via Pass@1 rather than relying on detection-and-repair loops.

## 4. Fine tuning using LoRA

**Input:** Hallucination-aware code samples, including buggy code, diagnostic information, and repair outcomes, along with a frozen pre-trained LLM.

**Output:** Client-specific LoRA adapters trained on hallucination aware datasets.

**Methods:**

- **Dataset Construction:** Construct structured dataset using problem description, buggy code, hallucination type, repairability flag and corrected code.
- **Low-Rank Decomposition:** Weight updates are parameterized using two low-rank matrices  $A$  and  $B$ , where the rank  $r$  is set to 8.
- **Attention Layer Injection:** LoRA modules are injected into the Query and Key projection matrices of each Transformer layer.
- **Parameter Isolation:** Original model weights remain frozen, and only LoRA parameters are made trainable.
- **Adapter Checkpointing:** Only trained LoRA adapter weights are saved and transmitted to the server, ensuring lightweight communication and privacy preservation.

## 5.Federated Aggregation and Distribution

**Input:** Client-side LoRA adapter weights,along with sample size.

**Output:**Global LoRA adapter distributed to all participating clients.

**Methods:**

- **Adapter Collection:** LoRA adapter weights are collected from all participating clients after local fine-tuning.
- **Federated Averaging:**Client-side adapters are aggregated using the Federated Averaging (FedErrorAvg) algorithm, which targets particulat type of error.
- **Global Adapter Construction:**A global LoRA adapter is constructed by averaging the weighted client adapters while keeping the base model parameters frozen.
- **Adapter Distribution:**The aggregated global LoRA adapter is redistributed to all clients and combined with the shared base model for subsequent inference or further training rounds.

# Formula for federated average

## FedAVG Function

$$w^{t+1} = \sum_{k=1}^K \frac{n_k}{N} w_k^t$$

K - client

$n_k$  = number of training samples on client  $k$

N – total number of samples used across all clients

## Error frequency

$$F_e = \sum_{k=1}^K count_{k,e}$$

## Weight definition

$$weight_e = \frac{F_e}{\sum_{e' \in errors} F_{e'}}$$

$F_e$  = total occurrences of error type ' $e$ ' across all clients

$count_{k,e}$  = number of errors of type ' $e$ ' in client  $k$

# Formulas for federated average

## Error Score

$$E_k = \sum_{e \in \text{errors}} \text{count}_{k,e} \cdot \text{weight}_e$$

For each client, the total count of each type of error along with its computed weight is accounted for

## Normalisation

$$\alpha_k = \frac{E_k}{\sum_{j=1}^K E_j}$$

## Augmented FedAvg

$$w^{t+1} = \sum_{k=1}^K \alpha_k w_k^t$$

$w_k^t$  - local model weight on round  $t$  on client  $k$

$\alpha_k$  - normalized weight assigned to client  $k$

$w^{t+1}$  - Global model weights after aggregation at round  $t + 1$

## 6.Evaluation and analysis

**Input:** Code generated by the base model and the federated LoRA-enhanced model across benchmark datasets.

**Output:** Benchmark-wise evaluation.

**Methods:**

- code-generation benchmarks including MBPP, HumanEval, DS-1000 are used here.
- **Functional Correctness (Pass@1):**The Pass@1 metric is used to measure functional correctness if code passes all unit tests on the first attempt.
- **Code Similarity Analysis:**CodeBLEU is used as a secondary metric to measure syntactic and structural similarity between generated code and reference implementations.
- **Before–After Comparison:**Model performance is compared before and after LoRA fine-tuning to quantify improvements in correctness and hallucination reduction.
- **FedAvg Vs FedErrorAvg :** To compare the hallucination reduction for both fed average and fed error average.

# IMPLEMENTATION DETAILS

## 1. Problem ingestion and code generation

```
from datasets import load_dataset
ds = load_dataset("openai/openai_humaneval")

*** README.md: 6.52k/? [00:00<00:00, 526kB/s]
openai_humaneval/test-00000-of-00001.par(...): 100% 83.9k/83.9k [00:00<00:00, 149kB/s]
Generating test split: 100% 164/164 [00:00<00:00, 3331.05 examples/s]

df_humaneval = pd.DataFrame(ds['test'])

print(df_humaneval.columns)

Index(['task_id', 'prompt', 'canonical_solution', 'test', 'entry_point'], dtype='object')
```

Loading HumanEval dataset

Loading MBPP dataset

Extraction function signature in MBPP  
which will be later used for testing.

```
dataset = load_dataset("google-research-datasets/mbpp")
sanitized_dataset = load_dataset("google-research-datasets/mbpp", "sanitized")

mbpp_df = sanitized_dataset['train'].to_pandas()

def extract_signature(code: str):
    for line in code.splitlines():
        line = line.strip()
        if line.startswith("def "):
            return line
    return None
mbpp_df['function_signature'] = mbpp_df['code'].apply(extract_signature)

print(mbpp_df.columns)

Index(['source_file', 'task_id', 'prompt', 'code', 'test_imports', 'test_list',
       'function_signature'],
      dtype='object')
```

# IMPLEMENTATION DETAILS

## 1.Problem ingestion

```
DS1000

▶ ds1k = load_dataset("xlangai/DS-1000")
  df_ds1k = ds1k['test'].to_pandas()
  df_ds1k['prompt_2'] = df_ds1k['prompt'].str.split('A:', n=1).str[0].str.strip()

... README.md: 100% ██████████ 554/554 [00:00<00:00, 67.6kB/s]
    test.jsonl: █████ 3.44M/? [00:00<00:00, 112MB/s]
    Generating test split: 100% ██████████ 1000/1000 [00:00<00:00, 17195.62 examples/s]

df_ds1k["task_id"] = [f"DS{str(i).zfill(3)}" for i in range(len(df_ds1k))]

print(df_ds1k.columns)

Index(['prompt', 'reference_code', 'metadata', 'code_context', 'prompt_2',
      'task_id'],
      dtype='object')
```

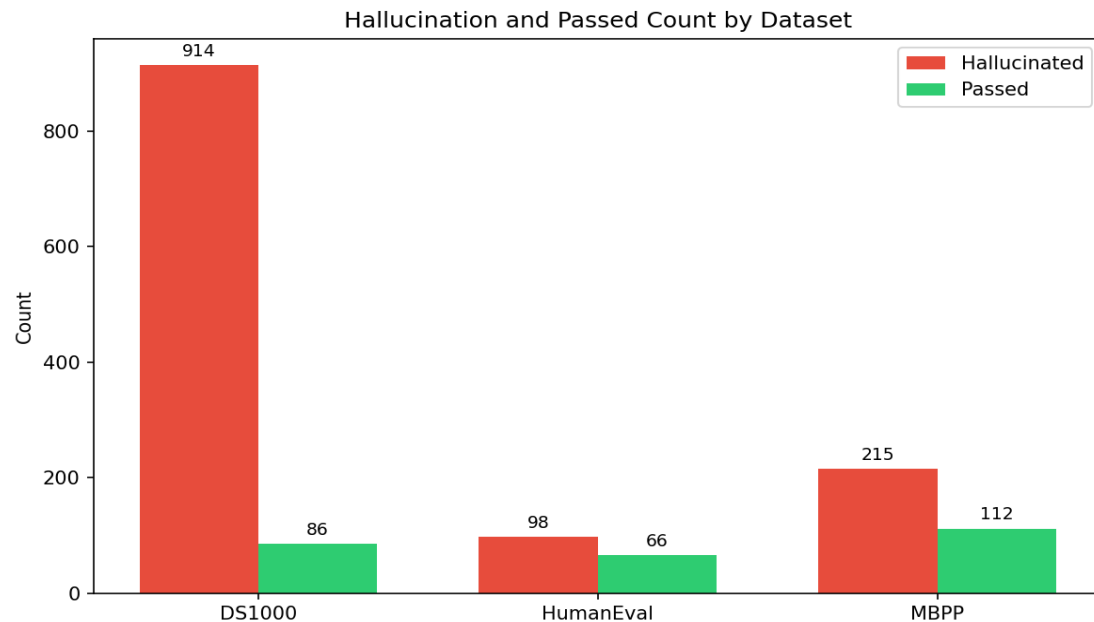
Loading DS1000 dataset

Created a column `task_id` to track data

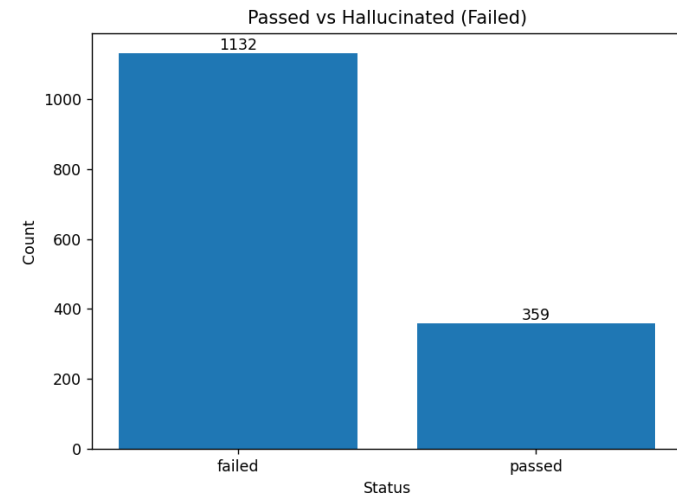


# IMPLEMENTATION DETAILS

## 2.Hallucination detection



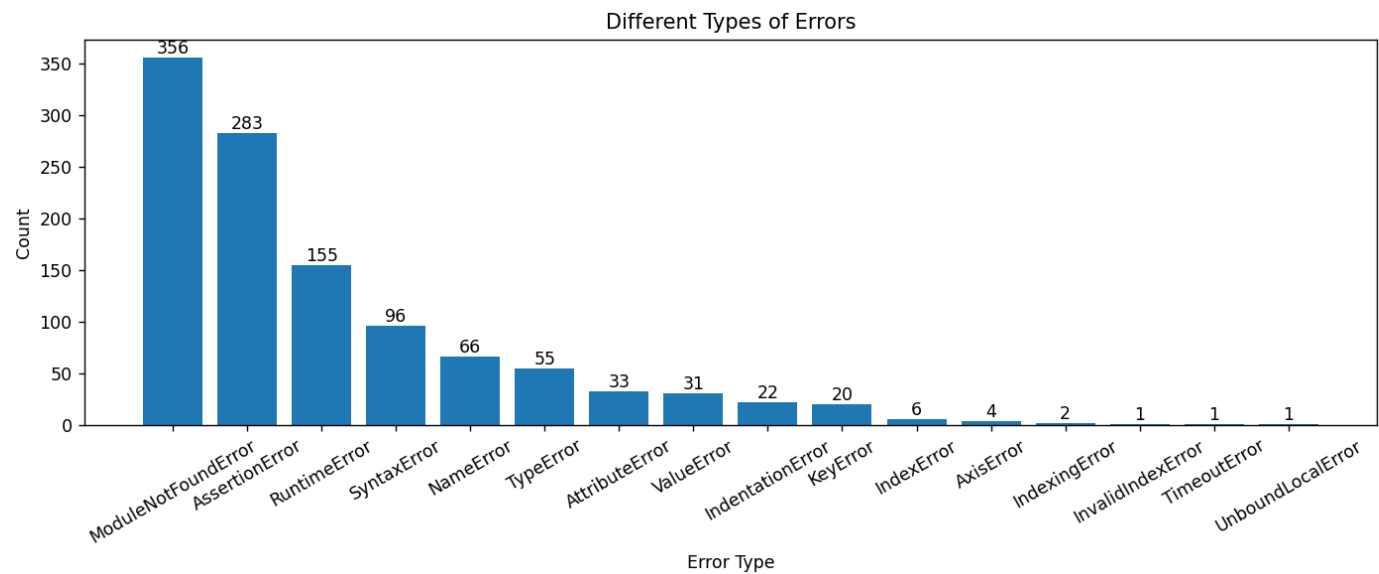
Hallucinated Vs Pass count across dataset



Overall pass and hallucinated code

# IMPLEMENTATION DETAILS

## 2.Dynamic Analysis



Types of hallucination at runtime

# IMPLEMENTATION DETAILS

## 3.Dataset creation

```
AST_SCHEMA = {
    "type": str | None,
    "value": int,
    "message": str | list[{
        "type": str,
        "line": int,
        "end_line": int
    }] | None
}
```

```
LIB_API_SCHEMA = {
    "type": "LibraryAPIError" | "ParsingError" | None,
    "value": int,
    "libapi_details": list[
        {
            "type": "module_not_found",
            "module": str,
            "line": int
        }
        |
        {
            "type": "name_error",
            "name": str,
            "line": int
        }
        |
        {
            "type": "attribute_error",
            "object": str,
            "attribute": str,
            "line": int
        }
    ]
}
```

```
DYNAMIC_SCHEMA = {
    "status": str,
    "error_type": str,
    "error_message": str,
    "line_number": str,
    "test_case": str, # JSON string
    "testcase_output": str,
    "generated_code": str,
    "dataset": str,
    "task_id": str
}
```

Fault information across different modules are collected for dataset creation and fine-tuning

# IMPLEMENTATION DETAILS

## 4.LoRA Fine Tuning

| Parameter      | Value                                                                         |
|----------------|-------------------------------------------------------------------------------|
| r (rank)       | 16                                                                            |
| lora_alpha     | 32                                                                            |
| target_modules | 7 projections (q_proj, v_proj, k_proj, o_proj, gate_proj, up_proj, down_proj) |
| lora_dropout   | 0.05                                                                          |
| bias           | "none"                                                                        |
| task_type      | CAUSAL_LM                                                                     |

LoRA Configuration for PEFT

# IMPLEMENTATION DETAILS

## 4.LoRA Fine Tuning

| Parameter                   | Value       |
|-----------------------------|-------------|
| num_train_epochs            | 3           |
| per_device_train_batch_size | 2           |
| gradient_accumulation_steps | 4           |
| learning_rate               | 2e-4        |
| weight_decay                | 0.01        |
| warmup_ratio                | 0.03        |
| lr_scheduler_type           | cosine      |
| max_grad_norm               | 0.3         |
| max_seq_length              | 2048        |
| packing                     | False       |
| bf16 / fp16                 | auto-detect |
| save_strategy               | epoch       |
| save_total_limit            | 1           |
| logging_steps               | 5           |

Configuration for SFT

# IMPLEMENTATION DETAILS

## 4.LoRA Fine Tuning

| === Attention projections === |               |              |              |             |
|-------------------------------|---------------|--------------|--------------|-------------|
| Module                        | W shape       | LoRA A shape | LoRA B shape | LoRA params |
| k_proj                        | [256, 2048]   | [16, 2048]   | [256, 16]    | 36,864      |
| o_proj                        | [2048, 2048]  | [16, 2048]   | [2048, 16]   | 65,536      |
| q_proj                        | [2048, 2048]  | [16, 2048]   | [2048, 16]   | 65,536      |
| v_proj                        | [256, 2048]   | [16, 2048]   | [256, 16]    | 36,864      |
| === MLP projections ===       |               |              |              |             |
| Module                        | W shape       | LoRA A shape | LoRA B shape | LoRA params |
| down_proj                     | [2048, 11008] | [16, 11008]  | [2048, 16]   | 208,896     |
| gate_proj                     | [11008, 2048] | [16, 2048]   | [11008, 16]  | 208,896     |
| up_proj                       | [11008, 2048] | [16, 2048]   | [11008, 16]  | 208,896     |

Adapters obtained after SFT

- The number of trainable parameters gets reduced from 2.7B to 29.9 M.
- Each targeted projection of layer get's two matrices A and B. Having 36 transformer layers and 7 projection per layer on decomposition we yield 504 adapters.

# IMPLEMENTATION DETAILS

## 5. Federated aggregation

---

**Algorithm 1** Fed-Error Average: Frequency-Weighted Federated Aggregation

---

**Require:** Client LoRA adapters  $\{w_k^t\}_{k=1}^K$ , error counts  $\{\text{count}_{k,e}\}$

**Ensure:** Global adapter  $w^{t+1}$

- 1: **Step 1:** Compute error frequency for each error type  $e$
  - 2:      $F_e \leftarrow \sum_{k=1}^K \text{count}_{k,e}$
  - 3: **Step 2:** Compute error-type weight
  - 4:      $\text{weight}_e \leftarrow \frac{F_e}{\sum_{e' \in \mathcal{E}} F_{e'}}$
  - 5: **Step 3:** Compute client error score  $E_k$
  - 6:      $E_k \leftarrow \sum_{e \in \mathcal{E}} \text{count}_{k,e} \cdot \text{weight}_e$
  - 7: **Step 4:** Normalize client weights
  - 8:      $\alpha_k \leftarrow \frac{E_k}{\sum_{j=1}^K E_j}$
  - 9: **Step 5:** Aggregate client adapters
  - 10:      $w^{t+1} \leftarrow \sum_{k=1}^K \alpha_k w_k^t$
  - 11: **return**  $w^{t+1}$
- 

Algorithm for Fed-error average

# IMPLEMENTATION DETAILS

## 5. Federated aggregation

```
Sample counts (n_k): {'mbpp': 327, 'humaneval': 164, 'ds1000': 1000}  
Weights (alpha_k): {'mbpp': 0.2193158953722334, 'humaneval': 0.10999329309188464, 'ds1000': 0.670690811535882}
```

*Figure 6.1.a: Output of fine-tuned adapters after Federated average (complete dataset)*

```
Final alpha_k: {'mbpp': 0.1765233238912576, 'humaneval': 0.039429434864408704, 'ds1000': 0.7840472412443337}
```

*Figure 6.2.a: Output of fine-tuned adapters after Federated average (Training dataset)*

```
Sample counts (n_k): {'mbpp': 245, 'humaneval': 123, 'ds1000': 750}  
Weights (alpha_k): {'mbpp': 0.21914132379248658, 'humaneval': 0.11001788908765653, 'ds1000': 0.6708407871198568}
```

*Figure 6.1.b: Output of fine-tuned adapters after Federated Error average (complete dataset)*

```
Final alpha_k: {'mbpp': 0.17893353222221403, 'humaneval': 0.04168543557403319, 'ds1000': 0.7793810322037528}
```

*Figure 6.2.b: Output of fine-tuned adapters after Federated Error average (Training dataset)*



# IMPLEMENTATION DETAILS

## 5. Verification

```
from transformers import AutoModelForCausalLM, AutoTokenizer
from peft import PeftModel
import torch
from tqdm import tqdm

base_id = "Qwen/Qwen2.5-Coder-3B-Instruct"
try:
    tokenizer = AutoTokenizer.from_pretrained(ADAPTER_PATH, trust_remote_code=True)
except Exception:
    tokenizer = AutoTokenizer.from_pretrained(base_id, trust_remote_code=True)
base_model = AutoModelForCausalLM.from_pretrained(base_id, device_map="auto", trust_remote_code=True)
model = PeftModel.from_pretrained(base_model, ADAPTER_PATH)
model.eval()
print("Model and tokenizer loaded.")
```

Code for model loading with adapters

- A key aspect of this evaluation framework is that the base model weights remain entirely frozen throughout the process. The federated adapter weights — aggregated LoRA matrices from across all clients — are applied as an additive overlay on top of the frozen base. This means the base model's pre-trained knowledge is preserved, and the adapter contributes only the task-specific delta learned during fine-tuning.
- The model is evaluated across all three datasets using four adapter variants: FedAvg Complete, FedAvg Train, FedErrorAvg Complete, and FedErrorAvg Train.

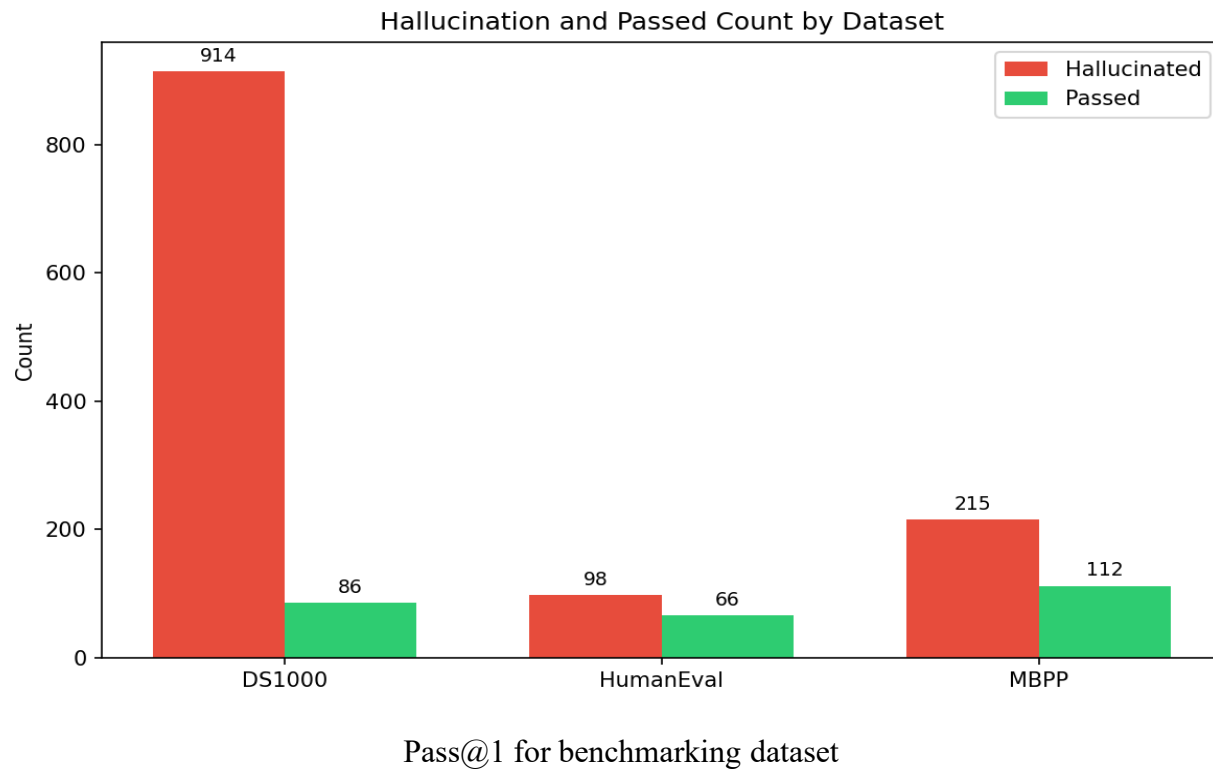
# EXPERIMENTAL SETUP

- **Base LLM :**
  - QwenCoder2.5-3B parameter is used as base model.
  - Type: Causal Language Models
  - Training Stage: Pretraining & Post-training
  - Number of Parameters: 3.09B
  - Number of Parameters (Non-Embedding): 2.77B
  - Number of Layers: 36
  - Number of Attention Heads (GQA): 16 for Q and 2 for KV
  - Context Length: Full 32,768 tokens
  - Deterministic Generation Settings: We configured `do_sample = False` and `temperature = 1.0` to ensure deterministic and reproducible code generation across multiple runs.

## EXPERIMENTAL SETUP

- **Dataset:**
  - **HumanEval:** Consists of 164 tasks focused on functional correctness.
  - **MBPP:** Consists of 377 tasks covering basic Python programming problems.
  - **DS-1000:** Consists of 1000 data science problems involving libraries like NumPy, Pandas, etc.
- **Development Environment:**
  - **Google Colab:** Used for LLM inference, code generation, and initial pipeline development; utilized NVIDIA T4 GPU with CUDA support.
  - **VS Code:** Used for local development of static and dynamic analysis modules, dataset processing, and result analysis; integrated with Git/GitHub for version control.
  - **JupyterHub (CTSKII):** Used for fine-tuning and federated learning experiments; provided NVIDIA A4000 GPU with persistent storage and containerized environment.

# RESULTS AND ANALYSIS

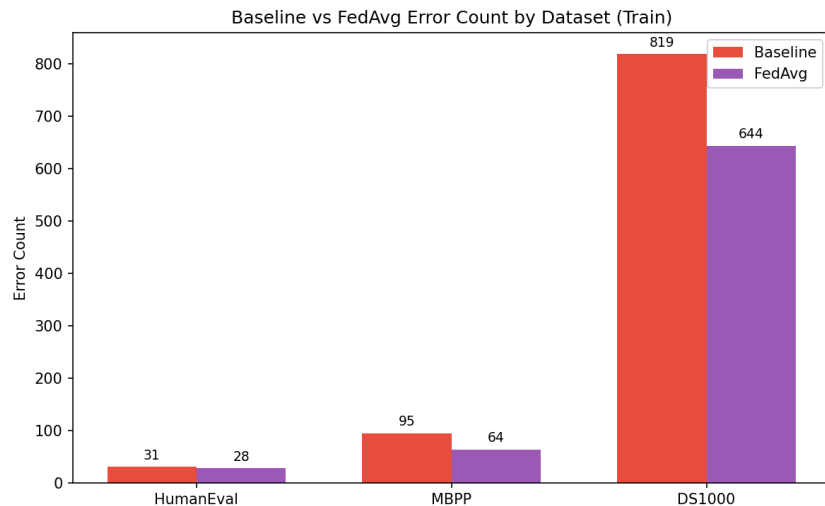


# RESULTS AND ANALYSIS

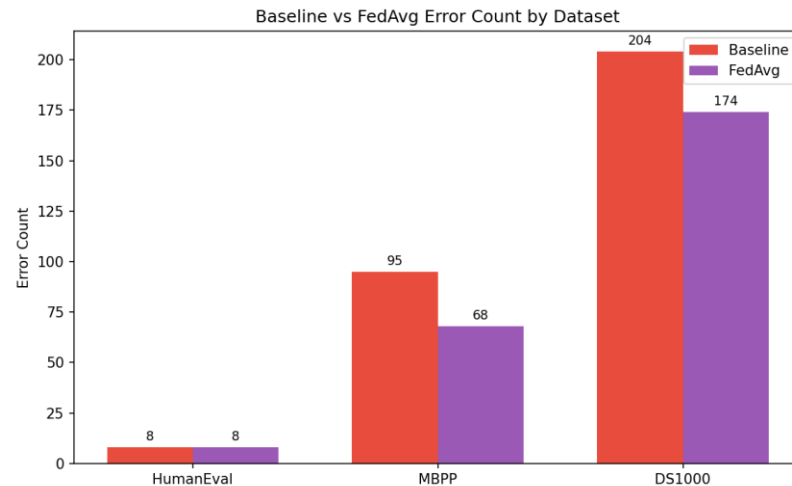
| Error_Type          | DS1000 | HumanEval | MBPP | Total |
|---------------------|--------|-----------|------|-------|
| AssertionError      | 188    | 20        | 75   | 283   |
| AttributeError      | 31     | 1         | 1    | 33    |
| AxisError           | 4      | 0         | 0    | 4     |
| IndentationError    | 90     | 0         | 0    | 90    |
| IndexError          | 4      | 1         | 1    | 6     |
| IndexingError       | 2      | 0         | 0    | 2     |
| InvalidIndexError   | 1      | 0         | 0    | 1     |
| KeyError            | 20     | 0         | 0    | 20    |
| ModuleNotFoundError | 352    | 3         | 1    | 356   |
| NameError           | 55     | 4         | 7    | 66    |
| RuntimeError        | 155    | 0         | 0    | 155   |
| SyntaxError         | 4      | 6         | 94   | 104   |
| TimeoutError        | 0      | 1         | 0    | 1     |
| TypeError           | 46     | 1         | 8    | 55    |
| UnboundLocalError   | 0      | 0         | 1    | 1     |
| ValueError          | 28     | 0         | 3    | 31    |
| attribute_error     | 22     | 0         | 0    | 22    |
| missing_return      | 3      | 53        | 66   | 122   |
| module_not_found    | 43     | 0         | 0    | 43    |
| name_error          | 1      | 52        | 0    | 53    |
| type_error          | 343    | 0         | 0    | 343   |
| Total               | 1392   | 142       | 257  | 1791  |

Types of errors with frozen model across all three dataset.

# RESULTS AND ANALYSIS

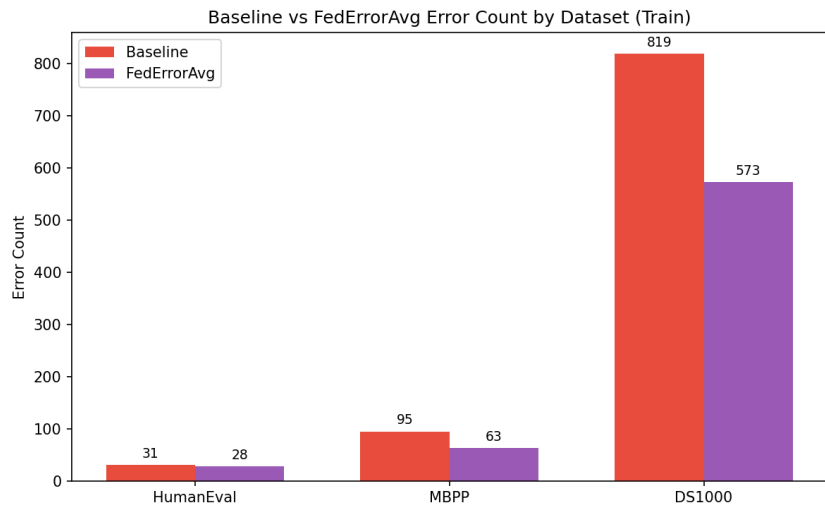


Hallucination count across all datasets after federated average, for the complete dataset

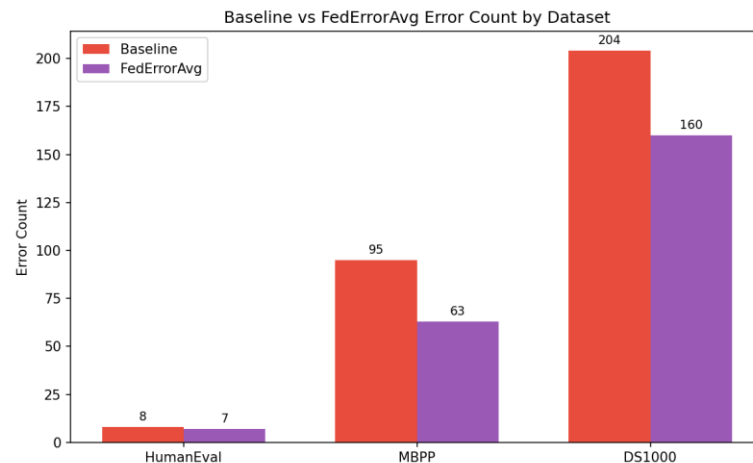


Hallucination count across all datasets after federated average, for the testing part of the dataset

# RESULTS AND ANALYSIS



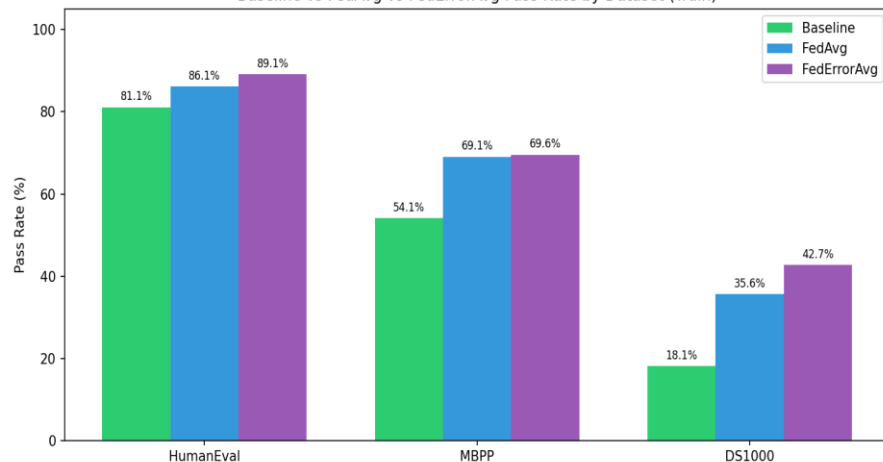
Hallucination count across all datasets after Fed-Error average, for the complete dataset



Hallucination count across all datasets after Fed-Error average, for the testing part of the dataset

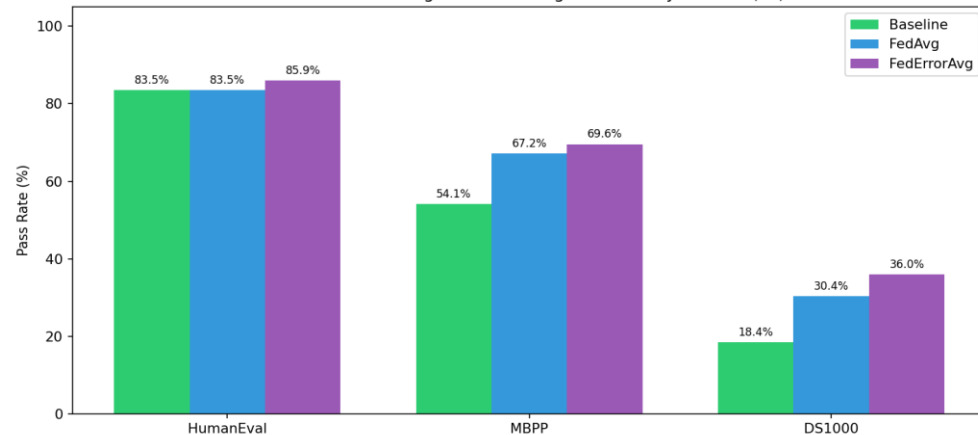
# RESULTS AND ANALYSIS

Baseline vs FedAvg vs FedErrorAvg Pass Rate by Dataset (Train)



Pass rate across all datasets after Fed-Error average,  
for the complete dataset

Baseline vs FedAvg vs FedErrorAvg Pass Rate by Dataset (TT)

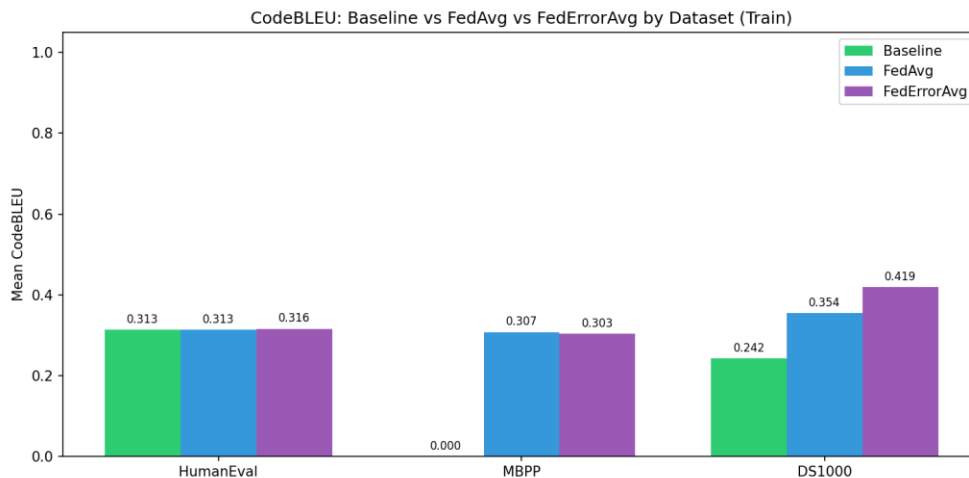


Pass rate across all datasets after Fed-Error average,  
for the testing part of the dataset



# RESULTS AND ANALYSIS

**CodeBLEU** is a metric for evaluating the quality of AI-generated code against reference code. It's an extension of the classic BLEU score but specifically designed for code.



CodeBLEU comparison between baseline, FedAverage and FedErrorAverage

## RESULTS AND ANALYSIS

| error_type        | baseline<br>_count | fedavg_<br>count | federrorav<br>g_<br>count | reduction |
|-------------------|--------------------|------------------|---------------------------|-----------|
| AssertionError    | 126                | 135              | 126                       | 0         |
| AttributeError    | 14                 | 5                | 7                         | 7         |
| AxisError         | 1                  | 2                | 1                         | 0         |
| FileNotFoundError | 1                  | 0                | 0                         | 1         |
| IndentationError  | 18                 | 16               | 16                        | 2         |
| IndexError        | 2                  | 4                | 5                         | 0         |
| InternalError     | 0                  | 5                | 5                         | 0         |
| InvalidIndexError | 1                  | 0                | 0                         | 1         |
| KeyError          | 5                  | 16               | 6                         | 0         |
| NameError         | 71                 | 17               | 13                        | 58        |
| NotFittedError    | 2                  | 2                | 0                         | 2         |
| RuntimeError      | 3                  | 2                | 3                         | 0         |
| SyntaxError       | 19                 | 9                | 10                        | 9         |
| TimeoutError      | 3                  | 0                | 0                         | 3         |
| TypeError         | 24                 | 17               | 18                        | 6         |
| UnboundLocalError | 1                  | 0                | 0                         | 1         |
| ValueError        | 16                 | 20               | 20                        | 0         |

*Error reduction comparison for Fed- Error average and baseline for test dataset*

| <b>error_type</b>       | <b>baseline_count</b> | <b>fedavg_count</b> | <b>federroravg_count</b> | <b>reduction</b> |
|-------------------------|-----------------------|---------------------|--------------------------|------------------|
| AssertionError          | 379                   | 364                 | 365                      | 14               |
| AttributeError          | 58                    | 43                  | 40                       | 18               |
| AxisError               | 4                     | 1                   | 0                        | 4                |
| FileNotFoundError       | 3                     | 0                   | 0                        | 3                |
| IndentationError        | 69                    | 48                  | 60                       | 9                |
| IndexError              | 6                     | 9                   | 9                        | 0                |
| IndexingError           | 2                     | 0                   | 0                        | 2                |
| InvalidIndexError       | 1                     | 0                   | 0                        | 1                |
| KeyError                | 19                    | 54                  | 18                       | 1                |
| LinAlgError             | 1                     | 0                   | 0                        | 1                |
| ModuleNotFoundError     | 2                     | 1                   | 2                        | 0                |
| NameError               | 201                   | 56                  | 40                       | 161              |
| NotFittedError          | 6                     | 5                   | 2                        | 4                |
| NotImplementedError     | 2                     | 1                   | 1                        | 1                |
| OverflowError           | 0                     | 1                   | 0                        | 0                |
| QhullError              | 0                     | 1                   | 1                        | 0                |
| RecursionError          | 0                     | 1                   | 0                        | 0                |
| RuntimeError            | 11                    | 7                   | 7                        | 4                |
| SyntaxError             | 24                    | 4                   | 4                        | 20               |
| TimeoutError            | 14                    | 0                   | 0                        | 14               |
| TypeError               | 80                    | 85                  | 84                       | 0                |
| UFuncTypeError          | 0                     | 4                   | 4                        | 0                |
| UnboundLocalError       | 1                     | 0                   | 0                        | 1                |
| UnidentifiedImageError  | 1                     | 0                   | 0                        | 1                |
| ValueError              | 61                    | 62                  | 55                       | 6                |
| return_outside_function | 0                     | 16                  | 6                        | 0                |

*Error reduction comparison for Fed- Error average and baseline*

## LITERATURE SURVEY

| S.NO | TITLE                                                                                   | NAME OF JOURNAL                                       | METHODOLOGY USED                                                                                                                                                                                                                                                                                                                       | LIMITATION                                                                                                                                                                                                                                                                |
|------|-----------------------------------------------------------------------------------------|-------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| [1]  | Advancing LLM-Generated Code Reliability: A Hybrid Approach for Hallucination Detection | IEEE Transactions on Software Engineering (TSE), 2025 | <ul style="list-style-type: none"><li>• Proposes SDHD, a hybrid hallucination detection framework that integrates static analysis and dynamic execution for LLM-generated code.</li><li>• This framework detects eight hallucination types and achieves superior precision, recall, and F1-score across multiple benchmarks.</li></ul> | <ul style="list-style-type: none"><li>• Dynamic detection depends on test coverage quality, and incomplete coverage may miss certain logical failures.</li><li>• The framework is currently validated only on Python, limiting cross-language generalizability.</li></ul> |

| S.NO | TITLE                                                                                  | NAME OF JOURNAL                                       | METHODOLOGY USED                                                                                                                                                                                                                                                                                                                                                                 | LIMITATION                                                                                                                                                                                                                                                   |
|------|----------------------------------------------------------------------------------------|-------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| [2]  | LLM-Based Test-Driven Interactive Code Generation: User Study and Empirical Evaluation | IEEE Transactions on Software Engineering (TSE), 2024 | <ul style="list-style-type: none"> <li>● TICODER is a test-driven interactive workflow that incrementally clarifies user intent through LLM-generated tests and lightweight user feedback.</li> <li>● The system ranks and prunes candidate code suggestions based on user-approved tests using two variants: PASS/FAIL validation and explicit output specification.</li> </ul> | <ul style="list-style-type: none"> <li>● There is interactive overhead, and incorrect or noisy user feedback can prune correct solutions prematurely.</li> <li>● Effectiveness is contingent on the LLM's ability to generate high-quality tests.</li> </ul> |

| S.NO | TITLE                                                                                    | NAME OF JOURNAL                               | METHODOLOGY USED                                                                                                                                                                                                                                                                                                                                                                                             | LIMITATION                                                                                                                                                                                                                                                                                         |
|------|------------------------------------------------------------------------------------------|-----------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| [3]  | A Federated Adaptive Large Language Model Fine-Tuning Framework for Software Development | IEEE Transactions on Services Computing, 2025 | <ul style="list-style-type: none"> <li>• The framework integrates LoRA-based parameter-efficient fine-tuning within a federated setting, where each organization locally trains lightweight adapter modules and shares only adapter to the central server.</li> <li>• A FedAvg-style is used to combine adapters based on local data size and an L2-norm constraint to manage gradient divergence</li> </ul> | <ul style="list-style-type: none"> <li>• Experimental results show degraded performance on unrelated datasets.</li> <li>• Although communication costs are reduced, the framework still requires multiple communication rounds, which may introduce latency in large-scale federations.</li> </ul> |

| S.NO | TITLE                                                                                                   | NAME OF JOURNAL                                | METHODOLOGY USED                                                                                                                                                                                                                                                                                                                                                                                         | LIMITATION                                                                                                                                                                                                                                                                                                        |
|------|---------------------------------------------------------------------------------------------------------|------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| [4]  | Towards Efficient Fine-Tuning of Language Models With Organizational Data for Automated Software Review | IEEE Transactions on Software Engineering 2024 | <ul style="list-style-type: none"> <li>Proposes CodeMentor, a framework for few-shot learning that utilizes heuristic rules and weak supervision to construct datasets from internal organizational data</li> <li>It employs Low-Rank Adaptation (LoRA) for parameter-efficient fine-tuning and integrates Reinforcement Learning from Human Feedback (RLHF) to incorporate domain expertise.</li> </ul> | <ul style="list-style-type: none"> <li>The framework depends on the quality of organizational data, which may introduce domain bias and limit generalization across different coding lang.</li> <li>RLHF requires computational resources and expert involvement, bottle neck scenario during scaling.</li> </ul> |

| S.NO | TITLE                                                              | NAME OF JOURNAL                                       | METHODOLOGY USED                                                                                                                                                                                                                                                                                                                                                                                                                   | LIMITATION                                                                                                                                                                                                                                                                                                                                                                                                        |
|------|--------------------------------------------------------------------|-------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| [5]  | The Impact of Prompt Programming on Function-Level Code Generation | IEEE Transactions on Software Engineering (TSE), 2025 | <ul style="list-style-type: none"> <li>● Empirical study to evaluate the impact of prompt programming techniques on function-level code generation.</li> <li>● CodePromptEval, a dataset of 7,072 prompts derived from 221 real-world Python functions, covering five prompt techniques: few-shot learning, chain-of-thought, persona, function signature, and package context, including all 32 possible combinations.</li> </ul> | <ul style="list-style-type: none"> <li>● The study is restricted to Python function-level tasks, limiting generalizability to other languages and higher-level program synthesis.</li> <li>● Rely on benchmark-driven evaluation, which may not fully capture real-world scenario.</li> <li>● Prompt techniques are evaluated in a fixed ordering, excluding effects of alternative prompt sequencing.</li> </ul> |



| S.NO | TITLE                                                                    | NAME OF JOURNAL                                | METHODOLOGY USED                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     | LIMITATION                                                                                                                                                                                                                                                                                                                                |
|------|--------------------------------------------------------------------------|------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| [6]  | An Adaptive Framework Embedded With LLM for Knowledge Graph Construction | IEEE Transactions on Multimedia, Vol. 27, 2025 | <ul style="list-style-type: none"> <li>● KG construction happens in three stages: Information Extraction (IE) for open triple extraction, Additional Relation (AR) for enriching triples with semantic information, and Knowledge Graph Normalization (KGN) for schema-level alignment using vector similarity search.</li> <li>● Adaptive Prompt Generation (APG) module automatically generates, evaluates, and ranks prompts to guide LLM behavior across stages, enabling multi-domain KG construction from text, speech, and images.</li> </ul> | <ul style="list-style-type: none"> <li>● High computational overhead due to multi-stage LLM inference, embedding generation, and vector similarity matching.</li> <li>● The framework relies heavily on prompt engineering, which, is not fully autonomous and may still suffer from hallucination and semantic inconsistency.</li> </ul> |

| S.NO | TITLE                                                                             | NAME OF JOURNAL                                                        | METHODOLOGY USED                                                                                                                                                                                                                                                                                                                                                                                                                                                      | LIMITATION                                                                                                                                                                                                                                                                                                                                                                                                   |
|------|-----------------------------------------------------------------------------------|------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| [7]  | Fight Fire With Fire: How Much Can We Trust ChatGPT on Source Code-Related Tasks? | IEEE Transactions on Software Engineering (TSE), Vol. 50, No. 12, 2024 | <ul style="list-style-type: none"> <li>Evaluating self-verification capability of GPT for: code generation, code completion, and program repair.</li> <li>Direct question, guiding question, and test report generation prompting is used assess whether ChatGPT can reliably validate its own outputs.</li> <li>Effectiveness is measured using accuracy, precision, recall, and F1-score, with additional analysis of self-contradictory hallucinations.</li> </ul> | <ul style="list-style-type: none"> <li>Results show that ChatGPT frequently overestimates the correctness of its own outputs, leading to low recall in direct self-verification.</li> <li>While guiding questions and test reports improve recall, precision is reduced.</li> <li>Generated test reports often contain incorrect explanations, especially for code generation and failed repairs.</li> </ul> |

| S.NO | TITLE                                                                                | NAME OF JOURNAL                                                 | METHODOLOGY USED                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              | LIMITATION                                                                                                                                                                                                                                                                                                                                                                          |
|------|--------------------------------------------------------------------------------------|-----------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| [8]  | FedALoRA: Adaptive Local LoRA Aggregation for Personalized Federated Learning in LLM | IEEE Internet of Things Journal, Vol. 12, No. 24, December 2025 | <ul style="list-style-type: none"> <li>Proposes FedALoRA, a personalized federated learning framework that integrates LoRA-based PEFT with adaptive local aggregation to address severe non-IID data in federated LLM training.</li> <li>FedALoRA uses an Adaptive LoRA Aggregation to merge global and local parameters via locally learned weights. This selective, element-wise fusion focused on higher transformer layers preserves domain expertise while capturing shared knowledge without increasing communication costs.</li> </ul> | <ul style="list-style-type: none"> <li>FedALoRA introduces additional local optimization complexity due to adaptive weight learning in ALoRA modules.</li> <li>The method requires careful tuning of the hyperparameter <math>l</math>, which controls the number of transformer layers involved in adaptive aggregation, and improper settings can degrade performance.</li> </ul> |

| S.NO | TITLE                                                                                 | NAME OF JOURNAL                                                            | METHODOLOGY USED                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       | LIMITATION                                                                                                                                                                                                                                                                                                                                                                                             |
|------|---------------------------------------------------------------------------------------|----------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| [9]  | No Need to Lift a Finger Anymore? Assessing the Quality of Code Generation by ChatGPT | IEEE Transactions on Software Engineering (TSE), Vol. 50, No. 6, June 2024 | <ul style="list-style-type: none"> <li>● Evaluation framework for assessing ChatGPT-based code generation. The authors construct standardized prompts for 728 LeetCode algorithm problems and 54 CWE security scenarios across five programming languages.</li> <li>● Generated code is evaluated using automated judgment platforms (LeetCode for functional correctness) and static analysis tools (CodeQL for vulnerability detection).</li> <li>● A multi-round fixing process is designed, where execution or analysis feedback is iteratively fed back to ChatGPT for up to 5 rounds.</li> </ul> | <ul style="list-style-type: none"> <li>● ChatGPT performs significantly better on problems published before 2021, indicating reliance on memorized patterns rather than reasoning.</li> <li>● The dialog-based fixing process has limited effectiveness, for logically complex problems .</li> <li>● Generated code often contains security vulnerabilities and non-deterministic behavior.</li> </ul> |

| S.NO | TITLE                                                   | NAME OF JOURNAL                                                                                | METHODOLOGY USED                                                                                                                                                                                                                                                                                                                                                                                                         | LIMITATION                                                                                                                                                                                                                                                                                        |
|------|---------------------------------------------------------|------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| [10] | Knowledge Enhanced Program Repair for Data Science Code | Proceedings of the 47th IEEE/ACM International Conference on Software Engineering (ICSE), 2025 | <ul style="list-style-type: none"> <li>• API KG Construction is done using a custom ontology and RDF triples to model API attributes and dependencies for seven data science libraries.</li> <li>• SPARQL is used for knowledge retrieval from on buggy code.</li> <li>• AST level Bug Knowledge Enrichment, which executes code node-by-node to localize runtime and assertion errors at the AST node level.</li> </ul> | <ul style="list-style-type: none"> <li>• DS-KG is version-specific and requires continuous updates to track evolving API documentation, making long-term maintenance costly and error-prone.</li> <li>• KG-based knowledge retrieval introduces higher latency than plain-text search.</li> </ul> |

| S.NO | TITLE                                                                 | NAME OF JOURNAL                                                                               | METHODOLOGY USED                                                                                                                                                                                                                                                                                                                                                                                                                                                            | LIMITATION                                                                                                                                                                                                                                                                                                                                    |
|------|-----------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| [11] | THINK:<br>Tackling API Hallucinations in LLMs via Injecting Knowledge | IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER), 2025 | <ul style="list-style-type: none"> <li>● THINK proposes a two-phase API hallucination mitigation framework consisting of Pre and Post execution Optimization.</li> <li>● Constructs an API knowledge database (Javadoc + validated code examples) and retrieves task-relevant APIs using task decomposition, similarity search, and LLM-based API labeling to guide code generation.</li> <li>● Error Classification into Seven API Error Types for correction .</li> </ul> | <ul style="list-style-type: none"> <li>● It may introduce irrelevant API knowledge, occasionally degrading performance for tasks already solvable by strong LLMs.</li> <li>● Both retrieval results and ReAct-based repair depend on LLM generation, leading to non-deterministic behavior and reduced reproducibility across runs</li> </ul> |

| S.NO | TITLE                                                                                                        | NAME OF JOURNAL                                            | METHODOLOGY USED                                                                                                                                                                                                                                                                                                                                                                                                                                                    | LIMITATION                                                                                                                                                                                                                                                                     |
|------|--------------------------------------------------------------------------------------------------------------|------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| [12] | EvoAPR: Enhancing Large Language Models for Automatic Program Repair with Genetic Algorithm and Dynamic LoRA | IEEE International Conference on Web Services (ICWS), 2025 | <ul style="list-style-type: none"> <li>Genetic evolution of buggy hunks under tri-objective fitness: syntax compliance, bug-hunk masking rate, repair-location masking rate.</li> <li>Dynamic LoRA swaps non-overlapping rank-r adapters per bug-index/project for on-demand, project-specific parameter retrieval.</li> <li>Base + LoRA models generate candidate patches; token-level entropy scorer picks the least uncertain fix as the final patch.</li> </ul> | <ul style="list-style-type: none"> <li>Genetic template evolution and dynamic LoRA fine-tuning introduce significant computation and implementation complexity.</li> <li>The effectiveness of both stages relies on accurate bug localization and rich bug context.</li> </ul> |

| S.NO | TITLE                                                                                | NAME OF JOURNAL                                                            | METHODOLOGY USED                                                                                                                                                                                                                                                                                                                                                                                                                                                        | LIMITATION                                                                                                                                                                                                                                                                                                      |
|------|--------------------------------------------------------------------------------------|----------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| [13] | CodeHalu: Investigating Code Hallucinations in LLMs via Execution Based Verification | Proceedings of the AAAI Conference on Artificial Intelligence (AAAI), 2025 | <ul style="list-style-type: none"> <li>Executes LLM-generated code against test cases under time and memory constraints, recording execution failures, unexpected outputs, infinite loop behaviors to detect hallucination states.</li> <li>Aggregates execution states across multiple LLMs and applies a two-stage heuristic induction process to classify hallucinations into four main types (Mapping, Naming, Resource, Logic) and eight subcategories.</li> </ul> | <ul style="list-style-type: none"> <li>The study focuses exclusively on Python, and hallucination behaviors in other programming languages are not evaluated.</li> <li>CodeHalu targets functional and logical correctness; identifying or mitigating security vulnerabilities is outside its scope.</li> </ul> |



# TIMELINE FOR COMPLETION OF MODULES

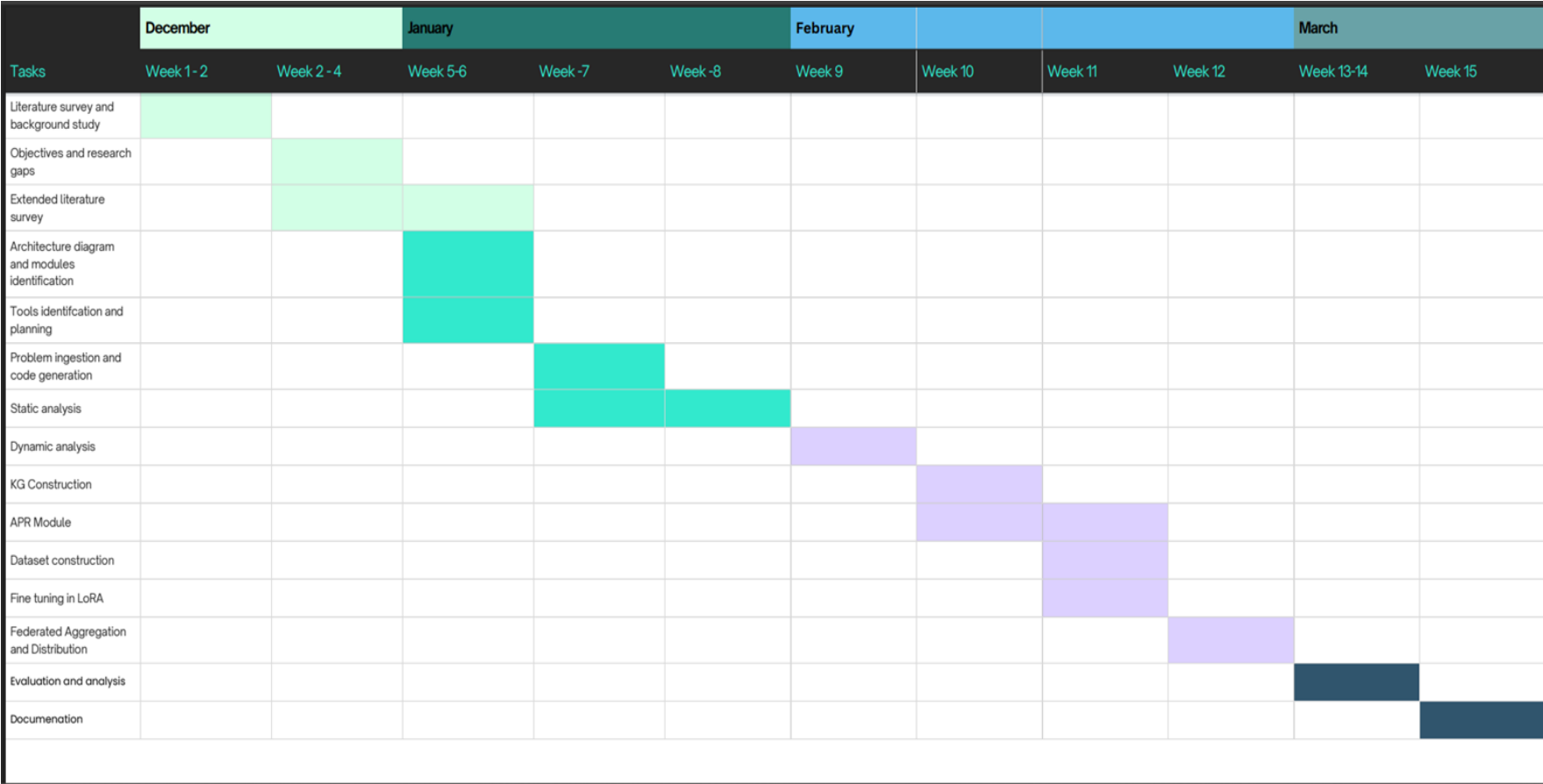


Fig.3 Federated learning + LoRA for fine-tuning

## REFERENCE

- [1] B. Yang, J. Dang, H. Liu and Z. Jin, "Advancing LLM-Generated Code Reliability: A Hybrid Approach for Hallucination Detection" in *IEEE Transactions on Software Engineering*, vol. , no. 01, pp. 1-17, PrePrints 5555, doi: 10.1109/TSE.2025.3640641.
- [2] S. Fakhoury, A. Naik, G. Sakkas, S. Chakraborty and S. K. Lahiri, "LLM-Based Test-Driven Interactive Code Generation: User Study and Empirical Evaluation," in *IEEE Transactions on Software Engineering*, vol. 50, no. 9, pp. 2254-2268, Sept. 2024, doi: 10.1109/TSE.2024.3428972.
- [3] J. Chen, Z. Cai, W. Chen, W. Wang, Z. Zheng and P. S. Yu, "A Federated Adaptive Large Language Model Fine-Tuning Framework for Software Development," in *IEEE Transactions on Services Computing*, doi: 10.1109/TSC.2025.3623626
- [4] M. Nashaat and J. Miller, "Towards Efficient Fine-Tuning of Language Models With Organizational Data for Automated Software Review," in *IEEE Transactions on Software Engineering*, vol. 50, no. 9, pp. 2240-2253, Sept. 2024, doi: 10.1109/TSE.2024.3428324
- [5] R. Khojah, F. G. de Oliveira Neto, M. Mohamad and P. Leitner, "The Impact of Prompt Programming on Function-Level Code Generation," in *IEEE Transactions on Software Engineering*, vol. 51, no. 8, pp. 2381-2395, Aug. 2025, doi: 10.1109/TSE.2025.3587794

## REFERENCE

- [6] Q. Wang, C. Li, Y. Liu, Q. Zhu, J. Song and T. Shen, "An Adaptive Framework Embedded With LLM for Knowledge Graph Construction," in *IEEE Transactions on Multimedia*, vol. 27, pp. 2912-2923, 2025, doi: 10.1109/TMM.2025.3557717
- [7] X. Yu, L. Liu, X. Hu, J. W. Keung, J. Liu and X. Xia, "Fight Fire With Fire: How Much Can We Trust ChatGPT on Source Code-Related Tasks?," in *IEEE Transactions on Software Engineering*, vol. 50, no. 12, pp. 3435-3453, Dec. 2024, doi: 10.1109/TSE.2024.3492204
- [8] X. Yi, C. Hu, B. Cai, H. Huang, Y. Chen and K. Wang, "FedALoRA: Adaptive Local LoRA Aggregation for Personalized Federated Learning in LLM," in *IEEE Internet of Things Journal*, vol. 12, no. 24, pp. 51854-51865, 15 Dec.15, 2025, doi: 10.1109/JIOT.2025.3582427
- [9] Z. Liu, Y. Tang, X. Luo, Y. Zhou and L. F. Zhang, "No Need to Lift a Finger Anymore? Assessing the Quality of Code Generation by ChatGPT," in *IEEE Transactions on Software Engineering*, vol. 50, no. 6, pp. 1548-1584, June 2024, doi: 10.1109/TSE.2024.3392499
- [10] S. Ouyang, J. M. Zhang, Z. Sun and A. M. Penuela, "Knowledge-Enhanced Program Repair for Data Science Code," in 2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE), Ottawa, ON, Canada, 2025, pp. 898-910, doi: 10.1109/ICSE55347.2025.00246.

## REFERENCE

- [11] J. Liu, Y. Zhang, D. Wang, Y. Li and W. Dong, "THINK: Tackling API Hallucinations in LLMs via Injecting Knowledge," *2025 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*, Montreal, QC, Canada, 2025, pp. 229-240, doi: 10.1109/SANER64311.2025.00029.
- [12] H. Zhang, Q. Yan, W. Min, C. Zhang, L. Kuang and Y. Xia, "EvoAPR: Enhancing Large Language Models for Automatic Program Repair with Genetic Algorithm and Dynamic LoRA," *2025 IEEE International Conference on Web Services (ICWS)*, Helsinki, Finland, 2025, pp. 946-956, doi: 10.1109/ICWS67624.2025.00123.
- [13] X. Yi, C. Hu, B. Cai, H. Huang, Y. Chen and K. Wang, "FedALoRA: Adaptive Local LoRA Aggregation for Personalized Federated Learning in LLM," in *IEEE Internet of Things Journal*, vol. 12, no. 24, pp. 51854-51865, 15 Dec.15, 2025, doi: 10.1109/JIOT.2025.3582427